

AIQT Milestone 12 Build Specification v0.2

Checkpoint and Graph Amendment Controls

Field	Value
Product	AIQT CLI
Milestone	M12
Version	v0.2
Status	Review candidate before build handoff
Depends on	M1-M11 complete; latest tag m11-issue-lifecycle-repair-controls
Primary user	Human project owner using AI coding agents
Primary agent target	Claude Code or equivalent coding agent

1. Source Alignment

M12 is a corrective recovery milestone after M11. M1-M11 established AIQT as a local state engine with canonical project/state files, append-only runlog history, bounded agent packets, checkpoint capture, review/export, manager reporting, issue lifecycle, and controlled repair-work promotion.

The governing workflow remains: **state -> graph -> packet -> checkpoint -> review/export -> manage/issues/repair**. M12 must not convert AIQT into a project manager SaaS, issue tracker replacement, AI runner, or graph database.

M12 addresses two historical dogfood incidents that previously required manual state editing: WU003 checkpoint classification recovery and DEP-031 dependency type correction. M9 solved acknowledgment, M10 solved classification, and M11 solved issue lifecycle. M12 adds controlled amendment commands for underlying checkpoint and graph records.

1.1 v0.2 Change Summary

- Adds an explicit checkpoint amendment status-transition table.
- Restricts amendment-triggered work-unit status changes to needs_review -> done only.
- Defines that done work units are not reopened or demoted by checkpoint amendments in M12.
- Adds a completion gate for needs_review -> done after effective checkpoint results are amended.
- Adds a unified overlay precedence model across canonical data, M12 amendments, M10 classification, and M11 issue overrides/promotion links.
- Requires graph validate to reuse existing M9/M10 graph warning logic where available rather than duplicating it.
- Adds explicit rationale for graph repair --dry-run only.
- Fixes complete JSON examples for every new command.

2. Milestone Objective

M12 adds safe, auditable controls for amending checkpoint interpretation and dependency modeling mistakes without hand-editing .aiqt/state.json. The milestone must preserve canonical history, append runlog events, recalculate derived workflow status, and expose dry-run/validation behavior before risky graph changes.

- Allow checkpoint result amendments as overlays, not destructive rewrites.
- Allow dependency type updates with cycle validation and readiness recalculation.
- Add graph validation that surfaces structural, cycle, readiness, and suspicious dependency findings.
- Add graph repair dry-run suggestions without automatic application.
- Preserve all M1-M11 behavior and tests.

3. Commands in Scope

```
aiqt checkpoint amend --checkpoint <checkpoint-id> --acceptance passed|failed|partial|not_checked --reason "..."  
aiqt checkpoint amend --checkpoint <checkpoint-id> --validation passed|failed|partial|not_run --reason "..."  
aiqt checkpoint amend --checkpoint <checkpoint-id> --acceptance passed --validation passed --reason "..."  
aiqt checkpoint amend ... --json
```

```
aiqt dependency update <dependency-id> --type blocks|requires|relates_to --reason "..."  
aiqt dependency update ... --json  
  
aiqt graph validate  
aiqt graph validate --json  
  
aiqt graph repair --dry-run  
aiqt graph repair --dry-run --json
```

4. Out of Scope

- No automatic graph repair apply command.
- No broad replanning or regeneration of the work graph.
- No deletion of checkpoints, dependencies, milestones, work units, or runlog records.
- No editing arbitrary checkpoint fields beyond validation/acceptance result overlays.
- No dependency create/delete commands.
- No checkpoint amendment that reopens done work or demotes terminal work.
- No issue lifecycle changes beyond respecting M11 issue states where relevant.
- No package publishing, skill installation, external API calls, or AI/agent execution.

5. Core Design Decision

M12 uses amendment overlays and auditable graph mutations, not historical rewrites. Checkpoint amendment changes the effective interpretation of a checkpoint while preserving the original checkpoint record. Dependency update changes the canonical dependency type, because the graph edge itself is the source of truth, but it must preserve an audit trail in runlog and validate graph integrity before mutation.

M12 is a controlled correction layer. It is not a rollback engine, reopen engine, broad replan system, or automatic repair executor.

6. Schema Contract

6.1 Schema version decision

M12 adds optional `state.checkpointAmendments`. This is additive and optional. Do not bump `AIQT_SCHEMA_VERSION` unless implementation proves unavoidable. Missing `checkpointAmendments` must be treated as an empty array.

6.2 Checkpoint amendment model

```
state.checkpointAmendments?: [  
  {  
    amendmentId: "AMEND-001",  
    checkpointId: "C003",  
    workUnitId: "WU003",  
    acceptanceCriteriaResult?: "passed" | "failed" | "partial" | "not_checked",  
    validationResult?: "passed" | "failed" | "partial" | "not_run",  
    reason: "...",  
    amendedAt: "ISO timestamp",  
    sourceCommand: "aiqt checkpoint amend"  
  }  
]
```

If multiple amendments exist for the same checkpoint/result field, the latest amendment by stored order is effective. If the existing project already has reliable ISO timestamp ordering helpers, they may be used, but stored order must remain deterministic. All amendments remain visible in manage/final-review where relevant.

6.3 Dependency update model

Dependency update mutates the existing `dependency.type` field only after validation. No separate dependency overlay is introduced in M12. The runlog event is the audit trail. The updated dependency remains canonical in `state.workGraph.dependencies`.

7. Unified Overlay and Classification Precedence

M12 adds a new effective-checkpoint overlay. The combined computation order must be explicit and shared by review, manage, issue list, repair plan, and final-review export.

Step	Layer	Rule
1	Canonical state data	Read project.json and state.json. Original checkpoints, work graph, dependencies, issue state, and review state remain the historical source.
2	M12 checkpoint amendments	Compute effective checkpoint validationResult and acceptanceCriteriaResult before deriving checkpoint-related review findings.
3	Graph validation/recalculation	Use canonical dependency types after any dependency update; reuse the existing readiness engine and current status transition helpers where possible.
4	M10 classification	Classify effective checkpoint findings and checkpoint issues into release/user-action/external/agent-fixable buckets.
5	M11 issue overrides	Apply issue lifecycle overrides such as accepted, deferred, resolved, and post_mvp after issue identities are derived.
6	M11 promotion links	Annotate issues that were promoted to repair work units and avoid duplicate repair recommendations.
7	Rendered outputs	review, manage, issue list, repair plan, graph validate, and final-review must render from the same effective model.

Operational rule: M12 amendments happen before review findings and issue classifications are derived. M11 overrides happen after issue identities are derived. Therefore an amended checkpoint can remove an active review finding entirely, while an M11 override keeps an issue visible but changes its lifecycle status.

8. Command Contract

8.1 aiqt checkpoint amend

Purpose: correct the effective validation or acceptance result of an existing checkpoint without rewriting the original checkpoint object.

- checkpoint ID must exist.
- at least one of --acceptance or --validation is required.
- --reason is required and non-empty after trim.
- amendment must not rewrite the original checkpoint record.
- review/manage/final-review must use effective amended checkpoint results.
- work unit, milestone, project status, dependency readiness, and nextRecommendedCommand must be recalculated if the amendment changes effective workflow state.
- append checkpoint.amended runlog event on mutation.
- idempotent no-op is allowed when the effective field values already match the requested values; return exit 0 and do not append a runlog event unless existing project convention logs no-ops.

8.2 aiqt dependency update

Purpose: safely correct dependency type mistakes such as relates_to -> blocks.

- dependency ID must exist.
- new type must be one of blocks, requires, relates_to.
- if type is unchanged, return exit 0 as idempotent no-op.
- for blocks/requires updates, validate that no cycle is introduced.
- recalculate work-unit readiness and milestone/project status.
- block with exit 2 if the update would invalidate an active in_progress execution cycle.
- append dependency.updated runlog event on mutation.
- do not create/delete dependencies.

8.3 aiqt graph validate

Purpose: read-only validation of graph structure and readiness semantics.

- Validate milestone/work-unit/dependency references.
- Detect cycles in blocking/requires dependency graph.

- Detect work units with invalid readiness state compared with dependencies.
- Detect suspicious late-stage relates_to patterns already identified in dogfood.
- Detect mixed inbound relates_to + blocks/requires warnings.
- Reuse existing M9/M10 graph warning implementation where available; do not duplicate or fork warning rules.
- If new graph validation rules are needed, add them to the shared graph/review warning service and cover them with tests.
- Do not mutate state or append runlog events.

8.4 aiqt graph repair --dry-run

Purpose: propose safe graph repairs without applying them.

- Requires --dry-run. No apply mode in M12.
- Use graph validate findings to propose candidate commands.
- Output copy-paste-runnable dependency update suggestions where deterministic.
- Do not mutate state, files, or runlog.
- If no deterministic repair exists, report investigation guidance only.

Rationale: M12 supports dry-run repair suggestions only because automatic repair application would introduce high-risk graph mutation. Apply mode is reserved for a later milestone after validation behavior is proven.

9. Effective Checkpoint Result Rules

Review rules, manage, final-review, issue list, and checkpoint-related findings must use effective checkpoint results. Effective result = latest amendment for that field if present, otherwise original checkpoint result.

This must fix the WU003-style case without deleting the original checkpoint history. If a done work unit originally has acceptanceCriteriaResult = partial but a later amendment sets it to passed, review must no longer report the original partial result as an active blocking finding. The amendment should remain visible as audit context.

10. Checkpoint Amendment Status Transition Rules

M12 must define whether checkpoint amendment can change workUnit.status. The only amendment-triggered terminal transition allowed in M12 is needs_review -> done. Done work must not be reopened or demoted by M12.

Current work unit status	May amend effective checkpoint result?	May change workUnit.status?	Behavior
done	yes	no	Recalculate review/manage/export findings only. Do not reopen, demote, or re-close the work unit.
needs_review	yes	yes, only to done	If completion gate passes, transition needs_review -> done, recalculate dependencies, and possibly unlock downstream work.
in_progress	no	no	Block with exit 2. Active execution cycle must be checkpointed first.
ready	no	no	Block with exit 2. No checkpointed execution to amend.
planned	no	no	Block with exit 2. No checkpointed execution to amend.
replanned	no	no	Block with exit 2 unless a future replanning model supports it.
cancelled	no	no	Block with exit 2. Cancelled work is not amendable in M12.

10.1 Completion gate for needs_review -> done

A needs_review work unit may become done through checkpoint amendment only when:

- effective validationResult = passed;
- effective acceptanceCriteriaResult = passed;

- the amended checkpoint is the latest checkpoint for that work unit;
- the checkpoint belongs to the same workUnitId stored on the work unit;
- no active currentWorkUnitId points at the same work unit.

Checkpoint issues remain visible through issue list, manage, and final-review unless separately overridden through M11 issue update. M12 does not auto-resolve checkpoint issues merely because validation and acceptance results become passed.

10.2 Non-transitioning amendments for done work

For done work units, checkpoint amendment may clear or change active review findings derived from validationResult or acceptanceCriteriaResult, but it must not alter workUnit.status. M12 does not support done -> needs_review, done -> planned, or any reopen flow.

11. Graph Recalculation Rules

After checkpoint amend or dependency update, AIQT must recalculate derived workflow state through the same service paths used by checkpoint, next-cancel, and promotion flows where possible. Do not create a second readiness engine.

- Work units with satisfied blocking/requires dependencies may become ready.
- Work units with newly unsatisfied blocking/requires dependencies must become planned unless they are done, cancelled, or otherwise terminal.
- A needs_review -> done amendment may unlock downstream planned work units.
- An in_progress current work unit must not be silently demoted by a dependency update; block the update with exit 2 if it would invalidate the active execution cycle.
- Project status, milestone status, currentMilestoneId, and nextRecommendedCommand must be recomputed consistently.
- nextRecommendedCommand remains exactly one command string.

12. Runlog Contract

All mutating commands append canonical runlog events using the existing RunlogEventSchema shape: id, type, timestamp, actor, summary, relatedIds, data.

12.1 checkpoint.amended

```
{
  "id": "EVT-...",
  "type": "checkpoint.amended",
  "timestamp": "2026-07-14T00:00:00.000Z",
  "actor": "cli",
  "summary": "Amended checkpoint C003 for work unit WU003",
  "relatedIds": ["C003", "WU003"],
  "data": {
    "amendmentId": "AMEND-001",
    "checkpointId": "C003",
    "workUnitId": "WU003",
    "acceptanceCriteriaResult": "passed",
    "reason": "Live Clerk verification was accepted as a user-action-required limitation.",
    "sourceCommand": "aiqt checkpoint amend"
  }
}
```

12.2 dependency.updated

```
{
  "id": "EVT-...",
  "type": "dependency.updated",
  "timestamp": "2026-07-14T00:00:00.000Z",
  "actor": "cli",
  "summary": "Updated dependency DEP-031 from relates_to to blocks",
  "relatedIds": ["DEP-031", "WU012", "WU023"],
  "data": {
    "dependencyId": "DEP-031",
    "fromWorkUnitId": "WU012",
    "toWorkUnitId": "WU023",
    "previousType": "relates_to",
    "newType": "blocks",
    "reason": "i18n readiness must wait for public profile work.",
    "sourceCommand": "aiqt dependency update"
  }
}
```

13. Exit Code Contract

Command	Exit 0	Exit 1	Exit 2	Exit 3
checkpoint amend	amendment stored or idempotent no-op	n/a	valid checkpoint exists but amendment is blocked by status or would create inconsistent workflow	unknown checkpoint, missing reason, invalid result, invalid state
dependency update	dependency updated or idempotent no-op	n/a	valid dependency exists but update is blocked by cycle, active execution, or invariant	unknown dependency, invalid type, missing reason, invalid state
graph validate	no blocking graph errors; warnings may exist	blocking graph integrity errors exist	n/a	invalid project/state
graph repair --dry-run	repair plan generated	n/a	no deterministic repair suggestions available but graph is readable	missing --dry-run, invalid project/state

M12 commands do not use exit code 10. Missing required arguments/options are invalid input and return exit 3.

14. JSON Output Contracts

14.1 checkpoint amend --json

```
{
  "status": "passed",
  "action": "checkpoint",
  "summary": "Checkpoint amendment stored.",
  "nextRecommendedCommand": "aiqt review",
  "data": {
    "amendmentId": "AMEND-001",
    "checkpointId": "C003",
    "workUnitId": "WU003",
    "effectiveAcceptanceCriteriaResult": "passed",
    "effectiveValidationResult": "passed",
    "workUnitStatusBefore": "needs_review",
    "workUnitStatusAfter": "done",
    "changed": true
  }
}
```

14.2 dependency update --json

```
{
  "status": "passed",
  "action": "dependency",
  "summary": "Dependency DEP-031 updated from relates_to to blocks.",
  "nextRecommendedCommand": "aiqt next",
  "data": {
    "dependencyId": "DEP-031",
    "previousType": "relates_to",
    "newType": "blocks",
    "affectedWorkUnits": ["WU023"],
    "changed": true
  }
}
```

14.3 graph validate --json

```
{
  "status": "warning",
  "action": "graph",
  "summary": "Graph validation completed with warnings.",
  "data": {
    "blockingErrors": [],
    "warnings": [
      {
        "rule": "mixed-inbound-dependency-types",
        "workUnitId": "WU025",
        "message": "Work unit has both relates_to and blocking inbound dependencies."
      }
    ]
  }
}
```

14.4 graph repair --dry-run --json

```
{
  "status": "warning",
  "action": "graph",
  "summary": "Graph repair dry-run generated candidate repairs.",
  "nextRecommendedCommand": "aiqt graph validate",
  "data": {

```

```

    "wouldMutate": false,
    "suggestions": [
      {
        "findingRule": "late-stage-relates-to",
        "dependencyId": "DEP-031",
        "recommendedCommand": "aiqt dependency update DEP-031 --type blocks --reason 'i18n readiness depends on WU012 public profile work'",
        "confidence": "medium"
      }
    ],
    "investigationGuidance": []
  }
}

```

15. Repository Implementation Scope

- src/cli/commands/checkpoint-amend.command.ts or existing checkpoint command subcommand.
- src/cli/commands/dependency-update.command.ts or dependency command module.
- src/cli/commands/graph-validate.command.ts and graph-repair.command.ts or graph command module.
- src/services/checkpoint-amendment-service.ts.
- src/services/graph-validation-service.ts.
- src/workflow/dependency-update-transition.ts.
- src/workflow/dependency-readiness.ts and existing recalculation helpers.
- src/schema/state.schema.ts and related amendment schemas.
- existing runlog-store helpers.
- tests/unit and tests/integration for all commands.

16. Runtime and Package Impact

- No new runtime dependency expected.
- No network calls.
- No AI/provider integration.
- No package publishing.
- All commands operate offline against local .aiqt state.
- graph validate and graph repair --dry-run are read-only and append no runlog events.

17. Test Matrix

17.1 Unit tests

- Effective checkpoint result uses latest amendment over original checkpoint.
- Checkpoint amendment rejects unknown checkpoint and empty reason.
- Checkpoint amendment status transition table is enforced for done, needs_review, in_progress, ready, planned, replanned, and cancelled work units.
- needs_review -> done only occurs when effective validationResult and acceptanceCriteriaResult are both passed.
- Done work units are not reopened or demoted by checkpoint amendments.
- Dependency update detects cycles before mutation.
- Dependency update idempotent same-type update returns success/no change.
- Graph validator detects broken references, cycles, stale readiness, mixed inbound dependency types, and late-stage relates_to warnings.
- Graph validate reuses shared warning rules where available.
- Graph repair dry-run emits deterministic suggested commands.

17.2 Integration tests

- WU003-style checkpoint amend removes active partial-checkpoint blocker while preserving amendment audit data.
- needs_review checkpoint amendment to passed/failed transitions the work unit to done and unlocks downstream work.
- done checkpoint amendment changes effective findings but does not reopen/demote work.
- DEP-031-style relates_to -> blocks update recalculates readiness and prevents premature ready state.
- Dependency update to blocks is blocked when it would invalidate an active in_progress work unit.
- graph validate read-only behavior: no state, export, or runlog mutation.
- graph repair --dry-run read-only behavior and deterministic output.
- Manage/final-review reflect effective checkpoint amendments and M11 issue overrides in the defined precedence order.

17.3 CLI contract tests

- Nested command --json parsing works.
- Missing --reason returns exit 3.
- Missing --dry-run for graph repair returns exit 3.
- Unknown checkpoint/dependency returns exit 3.
- Blocked graph mutation returns exit 2.
- No M12 command uses exit 10.
- All JSON examples are represented by complete, parseable command output fixtures where practical.

18. Definition of Done

- checkpoint amend implemented with additive amendments and no historical checkpoint rewrite.
- checkpoint amendment status-transition table implemented exactly, including needs_review -> done only.
- dependency update implemented with cycle validation and readiness recalculation.
- graph validate implemented as read-only structural and semantic validation using shared warning logic where possible.
- graph repair --dry-run implemented as read-only candidate repair planning.
- runlog events checkpoint.amended and dependency.updated use canonical event shape.
- manage, review, issue list, repair plan, and final-review use effective amended checkpoint results in the defined precedence order.
- exit codes follow the M12 contract.
- all existing tests pass.
- new tests cover WU003 and DEP-031 recovery scenarios.
- no package dependencies, external calls, docs artifacts, dist artifacts, or generated exports are committed.

19. Claude Code Handoff Summary

Implement M12 on a branch named **feat/m12-checkpoint-graph-amendment-controls**. Preserve AIQT as a local state engine. Do not broaden scope into package publishing or autonomous execution. Return an implementation report with files changed, schema changes, runlog events, validation results, tests added, deviations, and deferred items.